```
import java.sql.*;
import java.util.Scanner;
import java.math.BigDecimal;

public class Main {

    static Scanner sc = new Scanner(System.in);
    static Connection con;

    static String[] food = {
        "Veg Burger", "Veg Sandwich", "Margherita Pizza",
        "French Fries", "White Sauce Pasta", "Veg Momos",
        "Paneer Nuggets", "Veg Shawarma", "Cheese Maggi",
        "Paneer Roll"
    };

    static int[] price = {
        80, 50, 120, 60, 100, 70, 90, 100, 50, 80
    };

    static int[] prepTime = {
        3, 10, 15, 7, 12, 10, 12, 10, 5, 10
    };

    static boolean[] available = new boolean[10];

    public static void main(String[] args) {
```

```
try {
    System.out.print("Enter MySQL root password: ");
    String password = sc.nextLine();

    Class.forName("com.mysql.cj.jdbc.Driver");

    con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/queueless",
        "root",
        password
    );

    System.out.println("\nDatabase connected successfully!");

    resetAutoIncrementIfEmpty("OrderItems");
    resetAutoIncrementIfEmpty("Orders");
    resetAutoIncrementIfEmpty("Students");

    loadFoodMenu();

    int choice;

    do {
        System.out.println("\n========== QUEUELESS ==========");
        System.out.println("1. Student Menu");
        System.out.println("2. Staff Menu");
        System.out.println("3. Exit");
```

```
            System.out.print("Enter choice: ");

            choice = readInt();

            switch (choice) {
                case 1:
                    studentMenu();
                    break;

                case 2:
                    staffMenu();
                    break;

                case 3:
                    System.out.println("Thank you for using QueueLess!");
                    break;

                default:
                    System.out.println("Invalid choice!");
            }

        } while (choice != 3);

    } catch (ClassNotFoundException e) {
        System.out.println("MySQL Connector JAR not found!");
        System.out.println(e.getMessage());

    } catch (SQLException e) {
```

```
            System.out.println("Database error: " + e.getMessage());

        } finally {
            try {
                if (con != null) {
                    con.close();
                }
            } catch (SQLException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    static int readInt() {
        while (!sc.hasNextInt()) {
            System.out.print("Enter a valid number: ");
            sc.next();
        }

        int value = sc.nextInt();
        return value;
    }

    static void resetAutoIncrementIfEmpty(String tableName)
            throws SQLException {

        String sql = "SELECT COUNT(*) FROM `" + tableName + "`";
        long count;
```

```
    try (Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(sql)) {

        rs.next();
        count = rs.getLong(1);
    }

    if (count == 0) {
        String resetSql =
            "ALTER TABLE `" + tableName + "` AUTO_INCREMENT = 1";

        try (Statement st = con.createStatement()) {
            st.executeUpdate(resetSql);
        }
    }
}

static void loadFoodMenu() throws SQLException {

    String sql =
        "SELECT food_id, food_name, price, prep_time, availability " +
        "FROM FoodMenu";

    try (Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(sql)) {

        while (rs.next()) {
```

```
            int id = rs.getInt("food_id");

            if (id >= 1 && id <= food.length) {
                food[id - 1] = rs.getString("food_name");
                price[id - 1] =
                    rs.getBigDecimal("price").intValue();
                prepTime[id - 1] = rs.getInt("prep_time");
                available[id - 1] =
                    rs.getBoolean("availability");
            }
        }
    }
}

static void displayFoodMenu() {

    try {
        loadFoodMenu();
    } catch (SQLException e) {
        System.out.println("Unable to load menu: " + e.getMessage());
        return;
    }

    System.out.println("\n========== VEG FOOD MENU ==========");
    System.out.printf(
        "%-4s %-23s %-12s %-15s%n",
        "No.", "Food Item", "Price", "Availability"
    );
```

```
    for (int i = 0; i < food.length; i++) {
        System.out.printf(
            "%-4d %-23s Rs.%-9d %-15s%n",
            i + 1,
            food[i],
            price[i],
            available[i] ? "AVAILABLE" : "SOLD OUT"
        );
    }
}

static void studentMenu() {

    int choice;

    do {
        System.out.println("\n========== STUDENT MENU ==========");
        System.out.println("1. View Food Menu & Availability");
        System.out.println("2. Place Order");
        System.out.println("3. Check Order Status");
        System.out.println("4. Back");
        System.out.print("Enter choice: ");

        choice = readInt();

        switch (choice) {
            case 1:
```

```
                displayFoodMenu();
                break;

            case 2:
                placeOrder();
                break;

            case 3:
                checkStatus();
                break;

            case 4:
                break;

            default:
                System.out.println("Invalid choice!");
        }

    } while (choice != 4);
}

static void placeOrder() {

    displayFoodMenu();

    System.out.print("\nSelect food item (1-10): ");
    int selected = readInt();
```

```
if (selected < 1 || selected > food.length) {
    System.out.println("Invalid food selection!");
    return;
}

int index = selected - 1;

if (!available[index]) {
    System.out.println(food[index] + " is SOLD OUT!");
    return;
}

System.out.print("Enter quantity: ");
int qty = readInt();

if (qty <= 0) {
    System.out.println("Quantity must be greater than zero!");
    return;
}

sc.nextLine();

System.out.print("Enter student name: ");
String name = sc.nextLine().trim();

System.out.print("Enter student email: ");
String email = sc.nextLine().trim();
```

```
System.out.print("Enter student phone: ");
String phone = sc.nextLine().trim();

if (name.isEmpty()) {
   System.out.println("Student name cannot be empty!");
   return;
}

if (email.isEmpty()) {
   System.out.println("Student email cannot be empty!");
   return;
}

try {
   int studentId = findOrCreateStudent(name, email, phone);

   int token = getNextToken();

   int waitingTime =
      getPendingWaitingTime() + prepTime[index] * qty;

   con.setAutoCommit(false);

   try {
      int orderId;

      String orderSql =
         "INSERT INTO Orders " +
```

```
    "(token_number, student_id, order_status) " +
    "VALUES (?, ?, 'NEW')";

try (PreparedStatement ps =
         con.prepareStatement(
             orderSql,
             Statement.RETURN_GENERATED_KEYS
         )) {

    ps.setInt(1, token);
    ps.setInt(2, studentId);
    ps.executeUpdate();

    try (ResultSet keys = ps.getGeneratedKeys()) {
        if (!keys.next()) {
            throw new SQLException(
                "Unable to generate order ID."
            );
        }

        orderId = keys.getInt(1);
    }
}

String itemSql =
    "INSERT INTO OrderItems " +
    "(order_id, food_id, quantity, item_total) " +
    "VALUES (?, ?, ?, ?)";
```

```
try (PreparedStatement ps =
        con.prepareStatement(itemSql)) {

    ps.setInt(1, orderId);
    ps.setInt(2, selected);
    ps.setInt(3, qty);
    ps.setBigDecimal(
        4,
        new BigDecimal(price[index] * qty)
    );

    ps.executeUpdate();
}

con.commit();

System.out.println("\n========== ORDER CONFIRMATION ==========");
System.out.println("Order placed successfully!");
System.out.println("Order ID: " + orderId);
System.out.println("Student ID: " + studentId);
System.out.println("Token Number: " + token);
System.out.println("Food: " + food[index]);
System.out.println("Quantity: " + qty);
System.out.println(
    "Total Amount: Rs." + (price[index] * qty)
);
System.out.println(
```

```
                "Estimated Waiting Time: " +
                waitingTime + " minutes"
            );
            System.out.println("Order Status: Order Received");

        } catch (SQLException e) {
            con.rollback();
            throw e;

        } finally {
            con.setAutoCommit(true);
        }

    } catch (SQLException e) {
        System.out.println("Could not place order: " + e.getMessage());
    }
}

static int findOrCreateStudent(
    String name, String email, String phone
) throws SQLException {

    String findSql =
        "SELECT student_id FROM Students WHERE email = ?";

    try (PreparedStatement ps = con.prepareStatement(findSql)) {
        ps.setString(1, email);
```

```
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            return rs.getInt("student_id");
        }
    }
}

String insertSql =
    "INSERT INTO Students (student_name, email, phone) " +
    "VALUES (?, ?, ?)";

try (PreparedStatement ps =
         con.prepareStatement(
             insertSql,
             Statement.RETURN_GENERATED_KEYS
         )) {

    ps.setString(1, name);
    ps.setString(2, email);

    if (phone.isEmpty()) {
        ps.setNull(3, Types.VARCHAR);
    } else {
        ps.setString(3, phone);
    }

    ps.executeUpdate();
```

```
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) {
                return keys.getInt(1);
            }
        }
    }

    throw new SQLException("Could not create student!");
}

static int getNextToken() throws SQLException {

    String sql =
        "SELECT COALESCE(MAX(token_number), 100) + 1 " +
        "AS next_token FROM Orders";

    try (Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(sql)) {

        rs.next();
        return rs.getInt("next_token");
    }
}

static int getPendingWaitingTime() throws SQLException {

    int total = 0;
```

```
    String sql =
        "SELECT oi.food_id, oi.quantity " +
        "FROM Orders o " +
        "JOIN OrderItems oi ON o.order_id = oi.order_id " +
        "WHERE o.order_status IN ('NEW', 'PREPARING')";

    try (Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(sql)) {

        while (rs.next()) {
            int foodId = rs.getInt("food_id");
            int qty = rs.getInt("quantity");

            if (foodId >= 1 && foodId <= prepTime.length) {
                total += prepTime[foodId - 1] * qty;
            }
        }
    }

    return total;
}

static void staffMenu() {

    int choice;

    do {
        System.out.println("\n========== STAFF MENU ==========");
```

```
System.out.println("1. View All Orders");
System.out.println("2. Mark Order as READY");
System.out.println("3. Mark Order as COMPLETED");
System.out.println("4. Update Food Availability");
System.out.println("5. Back");
System.out.print("Enter choice: ");

choice = readInt();

switch (choice) {
   case 1:
      viewOrders();
      break;

   case 2:
      updateStatus("READY");
      break;

   case 3:
      updateStatus("COMPLETED");
      break;

   case 4:
      updateAvailability();
      break;

   case 5:
      break;
```

```
            default:
                System.out.println("Invalid choice!");
        }

    } while (choice != 5);
}

static void viewOrders() {

    String sql =
        "SELECT o.order_id, o.token_number, o.order_status, " +
        "fm.food_name, oi.quantity " +
        "FROM Orders o " +
        "JOIN OrderItems oi ON o.order_id = oi.order_id " +
        "JOIN FoodMenu fm ON oi.food_id = fm.food_id " +
        "ORDER BY o.order_id";

    System.out.println("\n========== ALL ORDERS ==========");

    try (Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(sql)) {

        System.out.printf(
            "%-8s %-8s %-23s %-10s %-15s%n",
            "OrderID", "Token", "Food", "Quantity", "Status"
        );
```

```
        int count = 0;

        while (rs.next()) {
            String status = rs.getString("order_status");

            if (status.equals("NEW")) {
                status = "PREPARING";
            }

            System.out.printf(
                "%-8d %-8d %-23s %-10d %-15s%n",
                rs.getInt("order_id"),
                rs.getInt("token_number"),
                rs.getString("food_name"),
                rs.getInt("quantity"),
                status
            );

            count++;
        }

        System.out.println("Total order items displayed: " + count);

    } catch (SQLException e) {
        System.out.println("Unable to view orders: " + e.getMessage());
    }
}
```

```
static void updateStatus(String newStatus) {

    System.out.print("Enter token number: ");
    int token = readInt();

    try {
        String findSql =
            "SELECT order_status FROM Orders WHERE token_number = ?";

        String oldStatus;

        try (PreparedStatement ps = con.prepareStatement(findSql)) {
            ps.setInt(1, token);

            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) {
                    System.out.println("Token number not found!");
                    return;
                }

                oldStatus = rs.getString("order_status");
            }
        }

        if (newStatus.equals("READY")) {
            if (oldStatus.equals("READY")) {
                System.out.println("Order is already READY!");
                return;
```

```
            }

            if (oldStatus.equals("COMPLETED")) {
                System.out.println("Order is already COMPLETED!");
                return;
            }
        }

        if (newStatus.equals("COMPLETED")
                && !oldStatus.equals("READY")) {
            System.out.println("Mark the order READY first!");
            return;
        }

        String updateSql =
            "UPDATE Orders SET order_status = ? WHERE token_number = ?";

        try (PreparedStatement ps = con.prepareStatement(updateSql)) {
            ps.setString(1, newStatus);
            ps.setInt(2, token);
            ps.executeUpdate();
        }

        System.out.println(
            "Token " + token + " status updated to " + newStatus + "!"
        );

    } catch (SQLException e) {
```

```
            System.out.println("Unable to update status: " + e.getMessage());
        }
    }

    static void updateAvailability() {

        displayFoodMenu();

        System.out.print("\nSelect food item (1-10): ");
        int selected = readInt();

        if (selected < 1 || selected > food.length) {
            System.out.println("Invalid food selection!");
            return;
        }

        System.out.println("\n1. Mark as AVAILABLE");
        System.out.println("2. Mark as SOLD OUT");
        System.out.print("Enter choice: ");

        int choice = readInt();

        if (choice != 1 && choice != 2) {
            System.out.println("Invalid choice!");
            return;
        }

        boolean newAvailability = choice == 1;
```

```
    String sql =
        "UPDATE FoodMenu SET availability = ? WHERE food_id = ?";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setBoolean(1, newAvailability);
        ps.setInt(2, selected);

        int changed = ps.executeUpdate();

        if (changed == 0) {
            System.out.println("Food item not found in database!");
            return;
        }

        available[selected - 1] = newAvailability;

        System.out.println(
            food[selected - 1] + " is now " +
            (newAvailability ? "AVAILABLE" : "SOLD OUT")
        );

    } catch (SQLException e) {
        System.out.println(
            "Unable to update availability: " + e.getMessage()
        );
    }
}
```

```
static void checkStatus() {

    System.out.print("Enter token number: ");
    int token = readInt();

    String sql =
        "SELECT o.token_number, o.order_status, fm.food_name, " +
        "oi.quantity " +
        "FROM Orders o " +
        "JOIN OrderItems oi ON o.order_id = oi.order_id " +
        "JOIN FoodMenu fm ON oi.food_id = fm.food_id " +
        "WHERE o.token_number = ?";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, token);

        try (ResultSet rs = ps.executeQuery()) {

            if (!rs.next()) {
                System.out.println("Token number not found!");
                return;
            }

            String status = rs.getString("order_status");

            System.out.println("\n========== ORDER DETAILS ==========");
            System.out.println("Token Number: " + rs.getInt("token_number"));
```

```
            System.out.println("Food: " + rs.getString("food_name"));
            System.out.println("Quantity: " + rs.getInt("quantity"));

            if (status.equals("NEW")) {
                System.out.println("Current Status: PREPARING");
                System.out.println("Your order is being prepared.");

            } else if (status.equals("PREPARING")) {
                System.out.println("Current Status: PREPARING");
                System.out.println("Your order is being prepared.");

            } else if (status.equals("READY")) {
                System.out.println("Current Status: READY");
                System.out.println("Please collect your order.");

            } else if (status.equals("COMPLETED")) {
                System.out.println("Current Status: COMPLETED");
                System.out.println("Order collected. Thank you!");

            } else {
                System.out.println("Current Status: " + status);
            }
        }

    } catch (SQLException e) {
        System.out.println("Unable to check status: " + e.getMessage());
    }
}
```

}